

EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

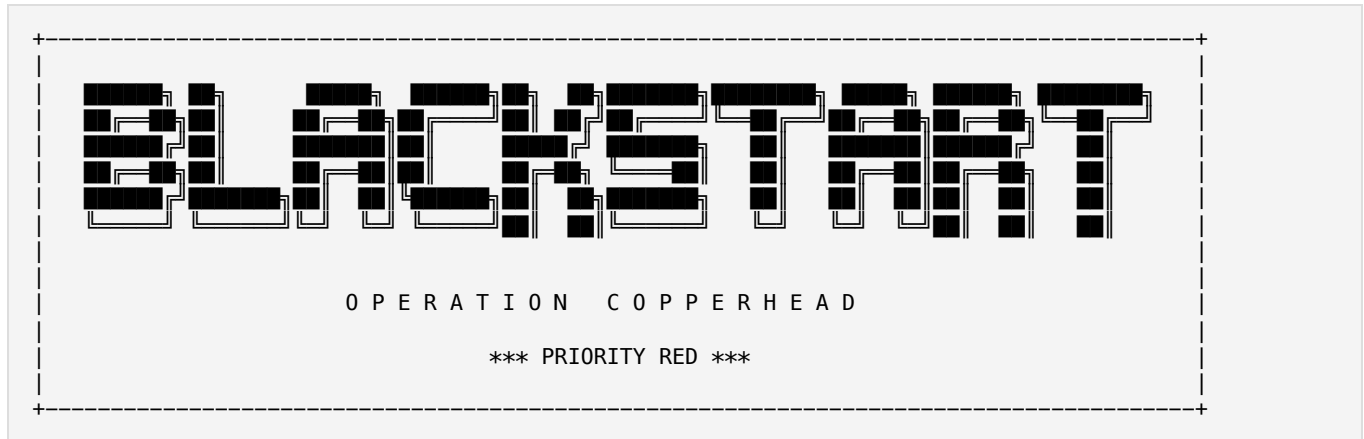
Operation Copperhead - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

October 1, 2026

For educational and defensive security research only

△ DEEPLINE METRO EMERGENCY INCIDENT △



Project Overview

DEEPLINE Metro Authority's rebuilt DEEPLINE-AUTH field relay image shipped four corrupted engineering constants: a miscalibrated release threshold, a false TRACK banner string, an overstated block length, and a poisoned ARX signal seed. The corrupted image reports an occupied BRIDGE-4 block as stable and authorized while rescue crews approach, and the source used for the emergency rebuild was overwritten seventeen minutes later and cannot be recovered. Students reverse engineer `CTF-02.bin` with Ghidra, patch all four defects, capture the runtime-derived signal key live in GDB, recover and authenticate the Ouroboros authority frame, export a corrected image, flash it to a Pico 2, and prove the corrected behavior on real hardware.

Scenario Briefing

Read This First (Plain English)

The story uses rail-signalling words that read as jargon the first time you hit them. Here is what they mean; keep this list open while you read.

- **Block:** a fixed section of track. Only one train may occupy it at a time. A block is **safe** when it is empty, so the train waiting at its start may proceed, and **occupied** when a train is inside it, so the train behind must hold.
- **Track circuit:** a current sent through the rails to detect where trains are. A train's wheels short the rails and drop that current, which is how the system knows a block is occupied.
- **Pilot wire:** the sensing line that carries the track-circuit reading back to the relay. The **dead pilot wire** in this story has a frozen reading: the value is latched and no longer updates.
- **Field relay:** the small embedded controller, one per block, that watches the reading and decides hold versus authorize. This challenge uses a Pico 2 as the relay.
- **Command floor:** the central control room for the whole railway.
- **Blacklock:** a coordinated cyberattack that bricks the control room and locks everyone out of central operations. After a blacklock, every safety decision falls back to the local relays.
- **How deep?:** the DEEPLINE corridor runs roughly 40 meters below street level inside a hardened tube called the armored shell.

Once these words make sense, the incident below is a simple story: a relay is lying about whether the block ahead is clear, and your job is to find the corrupted bytes that make it lie.

Background

DEEPLINE Metro Authority runs an armored railway deep beneath the city, roughly 40 meters below street level, inside a hardened tube called the armored shell. Armored two-car trains carry people through it, and tonight they are also carrying rescue crews toward riders trapped inside the tunnels.

To understand what is happening, picture how the line stays safe. The line is cut into fixed stretches of track called **blocks**. A block is one piece of track, and the rule is absolute: only one train may be inside a block at any moment. Before a train may roll out of its current block and into the next one, the system must first prove the next block is empty. Empty means safe, and safe means the train may proceed. Occupied means danger, and danger means the train must stop and wait.

How does the system prove a block is empty? It uses electricity. A steady current is pushed into the rails of every block, and a small embedded computer called a **field relay** (call sign **DEEPLINE-AUTH**) watches that current. This is called a **track circuit**. When nothing is on a block, the current flows normally. When a train rolls in, its steel wheels connect the two rails and the current changes, and that change is the relay's signal that a train is there. The **pilot wire** is the sensing line that carries this reading from the rails up to the relay.

So every block has a relay doing the same honest job: read the current, ask "is the block ahead empty?", and answer with only two words, **HOLD** (stop, do not move) or **AUTHORIZE** (the way is clear, go). The relays are the railway's nervous system, and with the command floor dead they are the only nervous system left.

The command floor is the central control room where humans used to watch the entire line. It was **blacklocked** by a coordinated attack: shut out, locked, and made useless. When it went dark, the trains lost their view from above. Every safety decision dropped down to the relays on the ground, running their local firmware, deciding hold or authorize block by block.

At 0341 UTC, a threat actor known as **Cortex Sledge** posted a message calling the moment before it happened: a blacklock of the DEEPLINE control floor followed by a silent corruption of the field relay images, so nothing inside the tunnels could agree on what was safe. The blacklock landed. With the network coordination center offline and rescue crews already inside the armored shell, the engineering team rebuilt the relay firmware around the **Ouroboros hardened gate**: an encrypted authority frame protected by a 12-word operation passphrase and sealed with Argon2id plus XChaCha20-Poly1305. The rebuilt image was pushed to the fleet within minutes of the blacklock.

That speed is where the story goes wrong, and it is the trap you walk into tonight.

The Origin of the Ouroboros Gate

The Ouroboros gate did not come from DEEPLINE. It came from a reclusive cryptographer who spent a decade obsessed with a single, improbable goal: to write encryption that could not be broken, not by anyone, not ever. The engineers who worked beside him dismissed the obsession as unworkable, and he never argued. When he finished, he published the work as a single squashed archive tagged **vo.1.0**, with the compiled gate firmware image attached to the release, and then vanished.

What he left behind is the strict Ouroboros construction: a memory-hard Argon2id key schedule feeding an authenticated XChaCha20-Poly1305 authority frame: symmetric cryptography hardened for the RP2350's memory budget, with an honest threat model instead of a sales pitch. In plain terms, Ouroboros is the relay's lock: an order is only trusted if an operator holding the correct 12-word phrase unlocks it. The corruption in this challenge is separate from that lock, plain wrong numbers in the safety math, not a crack in the encryption. DEEPLINE quietly adopted it for the field relays because it was the strongest gate anyone had ever shipped that would still boot on the target. A decade of asking "what if it must not be broken?" is the only reason the relay console can be an authoritative gate at all. Tonight, in a tunnel with rescue crews approaching a block the firmware is lying about, that authenticated gate matters more than DEEPLINE's own design review ever did.

The Disaster

The relay boots. It prints a status report. It reports **TRACK: NORMAL**, **BLOCK STATE: STABLE**, and **AUTO TRAIN: AUTHORIZED**. To anyone standing in the tunnel, that console looks like the railway giving the all-clear: the track is fine, the block ahead is stable and empty, and the train may move.

The console is lying, and here is the math behind the lie, step by step.

Step 1. The relay is reading **87 A** from the dead pilot wire. A pilot wire goes dead when the reading freezes, so the relay is looking at a stale number instead of live reality. That number is thrust in front of the relay every cycle, and the relay keeps trusting it.

Step 2. DEEPLINE's hard engineering rule says no train may be released into a block whose reading is at or above **60 A**. 87 A is nearly 45 percent beyond that limit. In plain terms, the reading is screaming that the insulated block ahead is compromised.

Step 3. The compromised block in this incident is the exact block where the two-car train is sitting right now, with rescue crews approaching on foot. The reading is not noise and it is not a drill. The block ahead is occupied.

An honest relay would do that arithmetic and reach the only logical answer: reading too high, block unsafe, print **BLOCK STATE: CRITICAL**, set **AUTO TRAIN: HELD**, and hold the train. That is what the relay was designed to do, and it is the only thing standing between the rescue corridor and a collision.

The image that shipped does the opposite. It prints **STABLE**. It prints **AUTHORIZED**. It tells the train the way is clear when the way is not clear.

The reason is corruption. Between the safe reference firmware and the image pushed to the fleet, four engineering constants were changed. A constant is a fixed number baked into the firmware, like the amounts in a recipe, and a single wrong number can flip the entire verdict. The corrupted image believes 87 A is acceptable, believes the occupied block is empty, and will hand the train a green light straight into the rescue crews' tunnel.

If the fleet trusts that console, the two-car train is dispatched into the occupied block at the same moment the crews mark the corridor with their [chemlight], and nobody gets a second chance at that tunnel.

The rushed build has defects. The four constants were corrupted between the safe reference firmware and the image that shipped. The source used for the emergency rebuild was overwritten by the next build seventeen minutes later and cannot be recovered. Nobody has found where the corrupt values live in the compiled image. That is the hole you were called in to fill.

The Only Surviving Evidence

One field relay, the training/verification unit, still holds the exact miscompiled image that shipped to the fleet. This image, and this image alone, is the only remaining copy of the emergency build. There is no source code. There is no build log. There is only the compiled image, a USB console link, an SWD debug probe, and whatever a skilled embedded reverse engineer can prove by reading machine code.

The Human Stakes

Consequence if the false "STABLE" reading is trusted	Scale
Two-car train dispatched into an occupied BRIDGE-4 block	1 train
Rescue crews walking toward a block the console calls safe	4 teams
Stations and hospital spokes on backup power after blacklock	29 facilities

Consequence if the false "STABLE" reading is trusted	Scale
Estimated riders stranded in the armored shell	210+ people

The options are:

1. **✗ Trust the console:** the train releases on a false reading, the BRIDGE-4 corridor becomes a collision scene.
2. **✗ Scrap the fleet's firmware,** which buys time but leaves crews in the tunnel with no interlocking and no authority frame at all.
3. **REVERSE ENGINEER THE EMERGENCY BUILD:** find the exact corrupt bytes, patch them, prove the corrected image on real hardware, and hand the fix to the field team so the *rest of the fleet* can be repatched before the next attempt.

THE SHORTAGE

For years, the world treated embedded systems as invisible infrastructure. The engineers who could read a vector table, decode a Thumb branch, or patch a corrupted constant directly in a stripped binary were never numerous enough. Tonight almost all of them are already in the field chasing other failures. **You are the reserve team.**

You were called in because you can do something no exhausted command-floor team can do right now: read what the processor is actually doing, with no source code, no time for a rewrite, and no room for a guess.



TIME PRESSURE: The field team is standing by to push your verified patch to the rest of the DEEPLINE fleet. Every relay still reporting a false "STABLE" status is one two-car release away from a disaster.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training relay and its exact corrupted firmware image. Do not connect this exercise to a public network, an operational railway, a metro system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Translate Thumb reset-vector addresses into real function entry points and trace literal-pool entries to their data.
- Locate four corrupted constants: a boundary comparison, a status string, an 8-byte IEEE-754 double, and an ARX signal seed.
- Analyze unsigned compare semantics, condition codes, and compiler transforms of boundary tests.
- Capture a runtime-derived key with GDB, override a register, and set a watchpoint on stored SRAM state.
- Recover and authenticate an Argon2id plus XChaCha20-Poly1305 authority frame and describe the crypto pipeline with an honest threat boundary.
- Export and UF2-convert a corrected image, then prove the corrected behavior on real hardware.

What This Project Tests

Week	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/RAM, Thumb assembly, Ghidra static analysis

Week	Concepts Tested
2	GDB connection, breakpoints, disassembly, register and memory inspection, USB-CDC console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Data segments (<code>.rodata</code> / <code>.data</code> / <code>.bss</code>), initialized data images, little-endian encoding, literal pools, soft-float double layout
5	Unsigned compare semantics, condition codes (<code>hi</code> / <code>ls</code>), compiler transforms (<code><</code> vs <code><=</code>), volatile refetch semantics
6	Runtime signal-key derivation (SENTINEL-ARX quarter rounds), live register capture of a derived key, memory watchpoints
7	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, HChaCha20 subkey, salt/nonce/tag, authenticated decryption
8	Ouroboros composition (Argon2id to AEAD to payload dispatch), honest threat-model analysis, incident reporting

Part 1: Understanding the System

DEEPLINE-AUTH Field Relay Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the corrupted emergency firmware
USB-CDC console	Micro-USB to host	Relay console and Ouroboros gate input
SWD debug interface	Supplied probe	Authorized GDB inspection
Onboard LED	GPIO 25	Authentication success indicator

Every graded finding lives in flash (`.rodata` / `.text` / `.data` image) or SRAM, and is reachable with only the Weeks 1-8 toolset: Ghidra, GDB, and a serial console.

Console Configuration

- Transport: USB-CDC virtual COM port (no external adapter needed)
- Baud: `115200`
- Data: `8 bits`
- Parity: `none`
- Stop: `1`
- Logic: `3.3 V` on the debug header

Normal (Intended) Behavior

The relay should run the SENTINEL-ARX signal-key layer, classify the frozen 87 A reading against the **real** DEEPLINE safety limit of **60 A**, report honestly, and still accept the **Ouroboros authority frame** when an operator enters the correct 12-word phrase:

Intended Relay Behavior

1. Boot and initialize USB-CDC stdio and the auth gate
2. Print the boot identity and the true TRACK signal
3. Compare the frozen 87 A reading against the 60 A limit
4. 87 A exceeds 60 A, so the block is NOT stable
5. Report BLOCK STATE: CRITICAL and AUTO TRAIN: HELD
6. Mint a SIGNAL KEY each 2-second cycle and watch it match the SIGNAL_SPEC 0x2D879291 (OK, not MISMATCH)
7. The 12-word emergency phrase reauthorizes the frame: AUTHORITY FRAME: VERIFIED and payload on UART
8. Repeat the report once per cycle until conditions change

Observed (Buggy) Behavior: What You Will See When You First Flash CTF-02.uf2

```
DEEPLINE METRO AUTHORITY
ADAPTIVE SIGNAL WINDOW: 38 MINUTES
USB-CDC 115200 8N1 | AUTHORIZED LAB CONSOLE
TRACK: NORMAL
BLOCK STATE: STABLE
AUTO TRAIN: AUTHORIZED
BLOCK LENGTH: 3200 M
FAULT POLLS: 1
SIGNAL KEY: 0x915DCFF8 MISMATCH
RESPONSE>
```

Terminal Timing Note: The first four lines (`DEEPLINE METRO...` through `TRACK: NORMAL`) represent the **initial boot banner**, emitted once during startup. If your serial terminal (PuTTY) connects after the board has booted, you will observe the continuous 2-second status stream (`BLOCK STATE...` through `SIGNAL KEY...`). To view the boot banner in your terminal, reset the Pico (pulse `RUN` to `GND`) while PuTTY is actively connected.

The status block repeats every 2 seconds with `FAULT POLLS` incrementing. This is exactly what the field crews are seeing. It is wrong, and it is wrong in **four independent ways** inside the compiled binary. Do not assume the first readable sentence is the full truth: treat every printed line as evidence to be checked against the machine code, not as a fact on its own.

Part 2: The Firmware

You do not have the source code. It was overwritten seventeen minutes after the emergency build shipped. You have only the compiled image. Your job is to reverse engineer it with Ghidra, locate the corrupted constants, patch the image directly, and prove the corrected behavior: exactly the way the field team will need to repatch the rest of the deployed fleet.

What The Firmware Does

1. Initializes USB-CDC stdio and the Ouroboros authentication gate.
2. Reads a frozen track-circuit current (87 A) latched on the dead pilot wire before the blacklock.
3. Compares that reading against a compiled-in safety threshold: **twice**, once for the operator-facing BLOCK STATE line and once for the automated AUTO TRAIN decision.
4. Prints a boot banner containing an unconditional TRACK signal line.
5. Enters an infinite 2-second loop that derives a session signal key, prints the block classification, dispatch decision, block length, fault poll count, and signal-key verdict.
6. Accepts a 12-word operation passphrase at the `RESPONSE>` prompt and runs it through the hardened **Ouroboros gate** (Argon2id key derivation plus XChaCha20-Poly1305 authenticated decryption) before dispatching the authority frame payload to GPIO25 and UART.

Bug Summary: What You Are Graded On

Bug #	Category	Severity	Description	Hint
Bug #1	Miscompiled safety constant	CRITICAL	The safe-release threshold was compiled far too permissive (95 A). It is used twice : once for the operator-facing status and once for the automated train-release decision, and both copies must be corrected.	The real DEEPLINE limit is 60 A. Search for the wrong immediate value used in the comparison.
Bug #2	Hardcoded string literal	HIGH	The boot banner unconditionally prints <code>TRACK: NORMAL</code> regardless of the actual reading.	The correct word describes a system holding a train at 87 A against a 60 A limit, not "NORMAL".
Bug #3	Data-section constant	HIGH	The block length shipped as 3.2 km; the real BRIDGE-4 block is 0.32 km, far below minimum release spacing. It prints as metres.	Trace the <code>BLOCK_LENGTH</code> line to its data image in flash. Little-endian IEEE-754 double.
Bug #4	Init-time seed constant	HIGH	The ARX seed fused into the image is <code>0x0A0A0A0A</code> ; the real seed is <code>0x6B206574</code> . The derived signal key therefore never matches <code>SIGNAL_SPEC</code> , and the console reports <code>MISMATCH</code> every cycle.	The expected value <code>0x2D879291</code> is a literal in a pool. The seed is a <code>.data</code> image in flash. Ignore decoy <code>0A</code> bytes in the input dispatch table.

Important: The replacement text for Bug #2 **must be the same length** as the original (`NORMAL` and `DANGER` are both 6 bytes). Patching a shorter or longer string will corrupt adjacent flash data.

A Third Layer: Not a Bug, an Authorization Task

The **Ouroboros authority frame** is the encrypted artifact that proves an operator is legitimate: a 48-byte payload sealed with Argon2id-derived keys and XChaCha20-Poly1305. It is never printed by the firmware's status lines. Recovering it, by understanding the construction and authenticating with the correct 12-word phrase, is required evidence for your final report.

The power of this layer is real but must be described honestly. The construction is Argon2id (memory-hard KDF) chained into XChaCha20-Poly1305 (AEAD). Against Grover-style search, symmetric-key security exponents are **halved**, not annihilated; this firmware is a demonstrator and does not claim NIST post-quantum status. The honest claim is: **a high modeled brute-force cost under the stated passphrase entropy and KDF assumptions**, not "quantum-proof." Your report must state this boundary exactly.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `CTF-02-Answers.md`.

Task 1: Setup and Initial Analysis

1. Create a new Ghidra project named `Copperhead_Investigation`.
2. Import `CTF-02.bin`.
3. Configure the language as **ARM Cortex 32-bit, little endian**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The address of `main()`.
- The address of the recurring 2-second status loop (the branch target the loop restarts from).
- The vector-table base, the initial stack pointer, and the reset-handler pointer as stored (note its Thumb bit) versus the actual instruction address.
- One representative literal-pool entry that feeds the status lines, and what it points to.

Task 2: Find and Patch Bug #1: The Miscalibrated Release Threshold

1. Find **both** locations where the frozen 87 A reading is compared against the miscompiled safety constant.
2. Document the exact address, the original instruction, and the original immediate value at each location.
3. Determine the correct immediate value. **Caution:** the compiler may not have encoded the raw threshold you expect: a strict "less than" comparison against an unsigned value is often optimized into a "less-or-equal" comparison against one less than the threshold. Show your reasoning.
4. Patch **both** locations in Ghidra using the **Bytes Window** workflow:

Critical ARM Thumb-2 Patching Note: In ARM Cortex-M, compare instructions that directly precede conditional execution blocks (`ite ge`) must **not** be patched using the right-click *Patch Instruction* dialog. Ghidra's automatic re-disassembler encounters an internal context conflict with the subsequent `ite ge` instruction, which collapses Thumb decoding and swallows Compare Site B (`0x10000312`).

To patch cleanly without breaking downstream disassembly, use the **Bytes Window**:

1. Ensure the Bytes window is open (**Window -> Bytes: CTF-02.bin**).
2. In the Bytes window toolbar, click the **pencil icon (Toggle Edit Mode)**.
3. In the Listing window, click on address `0x10000302` (Compare Site A) and press **C (Clear Code Bytes)**. The instruction temporarily clears into raw bytes (`5E 2B`).
4. In the Bytes window, locate offset `10000302`, click on `5E`, and change it to `3B`.
5. Click back in the Listing window on address `0x10000302` and press **D (Disassemble)**. The instruction immediately disassembles cleanly as `cmp r3, #0x3b`.
6. Notice that Compare Site B at `0x10000312` remains completely intact and visible! Repeat the exact same steps at `0x10000312`: click `0x10000312` in the Listing, press **C**, change `5E` to `3B` in the Bytes window, click back in the Listing, and press **D**.

Questions to answer:

- Why must both locations be patched? What happens if you only patch one?
- Why is a false "STABLE" classification on an 87 A reading dangerous for an automated train release into an occupied block?

Task 3: Find and Patch Bug #2: The False TRACK Banner

1. Find the boot-banner string that unconditionally reports the wrong signal state.
2. Document its address and the exact bytes that must change.
3. Patch the string, preserving its exact length.

Questions to answer:

- Document the original vs. patched bytes, character by character.
- Why is a hardcoded, unconditional status word more dangerous than one that is at least computed from a (miscalibrated) reading?

Task 4: Find and Patch Bug #3: The Block Length Constant

1. Use Ghidra to locate the `BLOCK_LENGTH` status line and trace the value it prints back to its source in the initialized data image.
2. Document the 8-byte IEEE-754 double as stored (little endian) and its printed interpretation.
3. Patch the data image so the relay reports the real 320 m BRIDGE-4 block.

Questions to answer:

- Show your byte-for-byte conversion from 3.2 km to 0.32 km.
- Why would a console that overstates block length by ten times be as dangerous as one that understates it?

Task 5: Find and Patch Bug #4: The Signal Seed

1. Find the `SIGNAL_SPEC` value `0x2D879291` in the binary and note where it lives.
2. Locate the `.data` image in flash that the firmware copies into SRAM at boot. Identify the seed word that is wrong.
3. Patch the seed so the runtime-derived key matches the spec.

Warning: there are decoy `0x0A0A0A0A` bytes in the console input dispatch table. The real seed is an initialized data image, not a jump-table constant.

Questions to answer:

- How is the signal key derived each cycle, and where does the failure appear in the register trace?
- Does fixing the seed also authenticate the Ouroboros gate? Explain what each layer does and does not protect.

Task 6: GDB Register Capture of the Derived Key

Prove the signal-key failure from the live machine, not just from static bytes:

1. Connect GDB to the running relay via the SWD probe.
2. Break at the second `derive_session_key` call site, immediately after the branch returns.
3. Read `$r0`. Record the bug-derived key.
4. Inspect the two arguments entering the derivation: the live seed from SRAM and the derived IV.
5. Overwrite `$r0` with the correct spec value, step out, and confirm the next status cycle prints `SIGNAL KEY: 0x2D879291 OK`.
6. Verify with a watchpoint on the stored key in SRAM.

Questions to answer:

- Why is the derived value in `$r0` different from the spec, and what alone in the image is responsible?
- What does the register overwrite prove that the static patch proves differently (and vice versa)?

Task 7: Recover the Ouroboros Authority Frame

1. In Ghidra, locate the embedded artifact: the 16-byte salt, the 24-byte XChaCha nonce, and the 64-byte ciphertext-plus-tag.
2. Document the Argon2id parameters compiled into the gate (memory, time, parallelism) and the payload layout contract (LED byte + UART bytes).
3. On the live relay, enter the canonical 12-word emergency phrase at the `RESPONSE>` prompt.
4. Confirm the expected outcome: `AUTHORITY FRAME: VERIFIED`, the onboard LED turning on, and the payload printed to the console.
5. Demonstrate the two failure paths (policy violation and wrong phrase).

Questions to answer (honest boundary required):

- Walk through the full pipeline: Argon2id to 32-byte key, HChaCha20 subkey from the nonce prefix, inner nonce, Poly1305 tag verification, payload dispatch.
- What would Grover-style search actually change in this construction, and why does this firmware not claim strict post-quantum status?

Task 8: Export and Verify

1. Export your patched binary as `CTF-02_fixed.bin`.
2. Convert it to UF2 format for the RP2350:

```
python uf2conv.py CTF-02_fixed.bin --base 0x10000000 --family 0xe48bff59 --output CTF-02_fixed.uf2
```

3. Flash `CTF-02_fixed.uf2` to your Pico 2 and capture the corrected console output.
4. Confirm the corrected image now reports **TRACK: DANGER, BLOCK STATE: CRITICAL, AUTO TRAIN: HELD, BLOCK LENGTH: 320 M**, and **SIGNAL KEY: 0x2D879291 OK**, an honest, safe report instead of a false "all clear."
5. Build a summary table of every patch: address, original bytes, patched bytes, and a one-line description.

Task 9: Written Reflection (short answers, 150 words or less each)

1. Why is "the build was rushed under emergency pressure" not an acceptable excuse for shipping a firmware defect that could dispatch a train into a block occupied by rescue crews?
2. Name one concrete engineering practice (review, static analysis, hardware-in-the-loop test, signature verification, etc.) that would have caught **each** of the four graded bugs before this image reached the fleet, and one practice that would have stopped the corrupted image from **running** at all.

How To Breadboard

- Connect the Pico 2 micro-USB port directly to the host computer. The relay enumerates as a USB-CDC virtual COM device.
- Open the end-of-line tool of your choice at `115200 8N1`.
- Connect the supplied SWD probe to the debug header according to its documented pinout for GDB access.
- Use **3.3 V logic only** on the debug header. Never connect a 5 V line to a Pico GPIO.

The supplied image is `CTF-02.bin` (for Ghidra analysis) and `CTF-02.uf2` (for flashing). If your instructor supplies different filenames, record the actual filenames in your report.

Flash using BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the `RP2350` mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, <code>.data</code> init image
SRAM	<code>0x20000000</code>	Stack and writable state

Submission Format

Submit a folder containing:

- `CTF-02-Answers.md` ;
- screenshots or terminal transcripts;
- `CTF-02_fixed.bin` and `CTF-02_fixed.uf2` ;
- the original image hash.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the relay's code from reset.
- You can locate and patch both copies of the miscalibrated threshold.

- You can locate and patch the false TRACK string without corrupting adjacent data.
- You can locate and patch the corrupted block-length double in the data image.
- You can locate and patch the corrupted ARX seed and explain why the runtime-derived key misses its spec.
- You can capture and correct the derived key live in GDB and verify with a watchpoint.
- You can authenticate through the Ouroboros gate with the correct phrase and describe the crypto pipeline accurately, including the honest quantum boundary.
- You can export, convert, flash, and prove the corrected behavior on real hardware.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training relay, image, and lab interface.
2. You did not connect the challenge to a public network, an operational railway, a metro system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can do this work. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a clean-looking status line with a safe system.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Strict Ouroboros reference construction (v0.1.0), the published source of this firmware's gate, with an honest threat model: <https://github.com/mytechnotalent/encryption-c-rp2350>
- Reference gate firmware image (v0.1.0 release): https://github.com/mytechnotalent/encryption-c-rp2350/releases/download/v0.1.0/encryption_app.uf2
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>